



INGENIERÍA DE SEGURIDAD US
Avda. Reina Mercedes s/n. 41012
Sevilla



DIGITALIZACIÓN DE LA IDENTIDAD Y LA GESTIÓN DE LA IDENTIDAD PARA UN SERVICIO DE SALUD PÚBLICO

Resumen Ejecutivo

Desde el equipo consultor de INSEGUS, valoramos la confianza que ha depositado en nosotros para resolver sus consultas tecnológicas referentes a la implementación de autenticación sin contraseñas mediante certificados digitales cualificados. Nuestro compromiso va más allá de ofrecerle estas soluciones puntuales y deseamos seguir contribuyendo a mejorar la seguridad de la información en su organización, alcanzando los objetivos de su negocio de forma segura.

En este trabajo, se ha diseñado una solución para implementar un sistema de autenticación sin contraseñas en el Servicio de Salud, utilizando certificados digitales cualificados para autenticar al personal. El proceso incluye:

- Generación del CSR y obtención del certificado de una CA.
- Verificación del control del dominio y la propiedad del par de claves.
- Implementación de la firma digital del nonce para autenticar al cliente sin contraseñas.
- Verificación de la validez del certificado usando protocolos como OCSP o CRL.

La implementación de esta solución garantizará una autenticación más segura, moderna y eficiente en el Servicio de Salud, mejorando la experiencia del usuario y reduciendo los riesgos asociados al uso de contraseñas tradicionales.

CONSULTA 3.1 SOLICITUDES DE CERTIFICADOS DIGITALES A LAS AUTORIDADES DE CERTIFICACIÓN CUALIFICADAS PARA LAS PARTES INTERESADAS Y ENVÍO A LAS AUTORIDADES DE CERTIFICACIÓN CUALIFICADAS.

3.1.1 GENERACIÓN DE LA SOLICITUD DE CERTIFICADOS DIGITALES CUALIFICADOS Y UTILIZACIÓN DE ESTOS POR LAS PARTES INTERESADAS DEL SERVICIO DE SALUD

1. Introducción

Nuestro objetivo consiste en la generación de una Solicitud de Certificado Digital Cualificado (CSR) siguiendo el estándar X.509 para su posterior uso en un Servicio de Salud Público, con el fin de facilitar la autenticación y firma electrónica de las partes interesadas en dicho sistema.

2. Descripción del Proceso de Generación del CSR

- **Generación de las Claves:**
 - **Par de Claves RSA:** Se genera un par de claves privada y pública con una longitud de 4096 bits, lo cual es un requisito de seguridad para garantizar la integridad de la información y la robustez del sistema de certificación.
 - **RSA** es un algoritmo asimétrico donde una clave (pública) se puede distribuir y otra (privada) se mantiene segura.
 - La clave privada nunca debe ser compartida, mientras que la clave pública se puede usar para autenticar la firma de la clave privada.

Código de Generación de Claves en Python:

```
consulta1 > consulta1.py > ...
1  from cryptography.hazmat.primitives.asymmetric import rsa
2  from cryptography.hazmat.primitives import serialization
3
4  private_key = rsa.generate_private_key(
5      public_exponent=65537,
6      key_size=4096,
7  )
8
9  # Guardar la clave privada
10 with open("private.key", "wb") as private_pem:
11     private_pem.write(
12         private_key.private_bytes(
13             encoding=serialization.Encoding.PEM,
14             format=serialization.PrivateFormat.PKCS8,
15             encryption_algorithm=serialization.NoEncryption()
16         )
17     )
18
19 public_key = private_key.public_key()
20
21 # Guardar la clave pública
22 with open("public.key", "wb") as public_pem:
23     public_pem.write(
24         public_key.public_bytes(
25             encoding=serialization.Encoding.PEM,
26             format=serialization.PublicFormat.SubjectPublicKeyInfo
27         )
28     )
29
```

3. Generación del CSR (Certificate Signing Request)

- **Definición del CSR:**
 - Un **CSR** es un archivo que contiene la **clave pública** y otros detalles relacionados con la identidad del solicitante (nombre, organización, etc.). Este archivo es firmado digitalmente con la clave privada para garantizar que quien lo envíe sea realmente el dueño de la clave privada.
- **Proceso de Generación del CSR:**
 - **Uso del Par de Claves:** Utilizando el par de claves generado previamente (clave privada y pública), se crea el CSR firmado con la clave privada.
 - **Información del Sujeto:** El CSR incluye la **información de identidad del solicitante**, como el nombre común (CN), organización (O), país (C), etc.
 - **Firma Digital:** Se firma el CSR utilizando la **clave privada** para demostrar que el solicitante es quien dice ser.

Código de Generación del CSR en Python:

- Menciona el código utilizado para crear el CSR y cómo se ha firmado digitalmente con la clave privada.

```
consulta1 > csr.py > ...
1 from cryptography.hazmat.primitives import hashes
2 from cryptography.x509 import CertificateSigningRequestBuilder, Name, NameAttribute
3 from cryptography.hazmat.primitives.asymmetric import rsa
4 from cryptography.hazmat.primitives import serialization
5 from cryptography.x509.oid import NameOID
6
7 # Cargar la clave privada
8 with open("private.key", "rb") as private_file:
9     private_key = serialization.load_pem_private_key(private_file.read(), password=None)
10
11 # Crear el sujeto del CSR
12 subject = Name([
13     NameAttribute(NameOID.COUNTRY_NAME, u"ES"),
14     NameAttribute(NameOID.STATE_OR_PROVINCE_NAME, u"Sevilla"),
15     NameAttribute(NameOID.LOCALITY_NAME, u"Sevilla"),
16     NameAttribute(NameOID.ORGANIZATION_NAME, u"Servicio de Salud"),
17     NameAttribute(NameOID.COMMON_NAME, u"Juan Perez"),
18     NameAttribute(NameOID.EMAIL_ADDRESS, u"juan.perez@serviciosalud.com"),
19 ])
20
21 # Crear el CSR
22 csr = CertificateSigningRequestBuilder().subject_name(subject).sign(private_key, hashes.SHA256())
23
24 # Guardar el CSR en un archivo
25 with open("request.csr", "wb") as csr_file:
26     csr_file.write(csr.public_bytes(serialization.Encoding.PEM))
27
28 print("CSR generado correctamente.")
29 |
```

4. Verificación de la Identidad del Solicitante

- **Métodos de Verificación:**
 - Para que un certificado digital cualificado sea válido, la **Autoridad de Certificación (CA)** debe verificar la identidad del solicitante. Esto puede hacerse mediante:

- **Presencia física:** Verificación del documento de identidad en persona.
- **Videollamada:** Confirmación de la identidad por medio de una videollamada.
- **Firma electrónica:** Si el solicitante ya tiene un certificado digital, puede firmar un documento con su clave privada para demostrar su identidad.
- **Biometría:** Uso de tecnología como reconocimiento facial o huellas dactilares.

5. Almacenamiento Seguro de la Clave Privada

- **Importancia de la Clave Privada:**
 - La clave privada debe ser protegida adecuadamente, ya que es la base para la firma de documentos y la autenticación.
- **Opciones para Guardar la Clave Privada de Forma Segura:**
 - **Hardware Security Module (HSM):** Dispositivos físicos diseñados para almacenar claves privadas de manera segura.
 - **Token USB:** Dispositivos portátiles que también pueden almacenar claves privadas.
 - **Almacenamiento Cifrado:** Si la clave se guarda en un archivo, debe estar cifrada y protegida con una contraseña fuerte.

Recomendación: La mejor opción sería almacenar la clave privada en un HSM o un Token USB para mayor seguridad.

6. Envío del CSR a la Autoridad de Certificación (CA)

- **Proceso de Envío del CSR:**
 - El archivo **CSR** generado se debe enviar a la **Autoridad de Certificación (CA)** para obtener el certificado digital cualificado.
 - El **envío** puede hacerse de dos formas:
 - **Correo Electrónico Seguro:** Utilizando PGP o S/MIME.
 - **Portal Web de la CA:** Subiendo el CSR a un portal web seguro proporcionado por la CA.
- **Verificación de la Identidad por la CA:**
 - Menciona que la **CA** realizará la **verificación de la identidad** del solicitante utilizando los métodos mencionados anteriormente.
- **Emisión del Certificado:**
 - Una vez que la identidad del solicitante ha sido verificada, la CA emitirá el certificado digital cualificado.

3.1.2 GENERACIÓN DE LA SOLICITUD A LAS AUTORIDADES DE CERTIFICACIÓN DE LOS CERTIFICADOS DIGITALES PARA LOS SERVIDORES WEB DE LA ENTIDAD Y VALIDACIÓN DEL CONTROL DEL DOMINIO Y POSESIÓN DEL PAR DE CLAVES.

- El objetivo es implementar un sistema de autenticación **sin contraseñas (passwordless)** utilizando certificados digitales cualificados para autenticar al personal del Servicio de Salud, utilizando Apache y Let's Encrypt (o cualquier otra Autoridad de Certificación).

Desafíos:

- **Verificación del control de dominio:** Asegurar que el servidor controla el dominio para el cual solicita el certificado.
- **Verificación de la propiedad del par de claves:** Demostrar que el servidor posee la clave privada del certificado digital.

1. Proceso de Solicitud de Certificado Digital para el Servidor Web

- **Generación del CSR**
 - **Herramienta utilizada: OpenSSL o Python con cryptography.**
 - **Proceso:** Se generó un **CSR** utilizando el nombre de dominio y la información relevante del servidor. Se utilizó el código en **Python** para generar las claves pública y privada, y luego el CSR.

```
1  from cryptography.hazmat.primitives.asymmetric import rsa
2  from cryptography.hazmat.primitives import serialization
3  from cryptography.x509 import CertificateSigningRequestBuilder, Name, NameAttribute
4  from cryptography.hazmat.primitives import hashes
5  from cryptography.hazmat.primitives.asymmetric import rsa
6  from cryptography.x509.oid import NameOID
7
8  private_key = rsa.generate_private_key(
9      public_exponent=65537,
10     key_size=4096,
11 )
12
13 with open("private_server2.key", "wb") as private_pem:
14     private_pem.write(
15         private_key.private_bytes(
16             encoding=serialization.Encoding.PEM,
17             format=serialization.PrivateFormat.PKCS8,
18             encryption_algorithm=serialization.NoEncryption()
19         )
20     )
```

```
# Crear el nombre del sujeto para el CSR (con el CN como el nombre del dominio)
subject = Name([
    NameAttribute(NameOID.COUNTRY_NAME, u"ES"),
    NameAttribute(NameOID.STATE_OR_PROVINCE_NAME, u"Sevilla"),
    NameAttribute(NameOID.LOCALITY_NAME, u"Sevilla"),
    NameAttribute(NameOID.ORGANIZATION_NAME, u"Servicio de Salud"),
    NameAttribute(NameOID.ORGANIZATIONAL_UNIT_NAME, u"Departamento de IT"),
    NameAttribute(NameOID.COMMON_NAME, u"www.serviciosalud.com"), # Nombre del dominio
])

csr = CertificateSigningRequestBuilder().subject_name(subject).sign(private_key, hashes.SHA256())

with open("server_request2.csr", "wb") as csr_file:
    csr_file.write(csr.public_bytes(serialization.Encoding.PEM))

print("CSR para servidor generado correctamente.")
```

Se generaron la clave privada y el CSR. El archivo `server_request.csr` es lo que se enviará a la CA para obtener el certificado.

2. Verificación del Control del Dominio

- La **CA** proporciona un archivo de desafío (por ejemplo, **fileauth.txt**) que contiene un código único. Este archivo debe ser colocado en un directorio específico del servidor web para confirmar que el servidor tiene control sobre el dominio.
- **Proceso de Colocación del Archivo de Desafío:**
El archivo **fileauth.txt** se coloca en el servidor web en el directorio **/.well-known/pki-validation/**.
- Usando **Docker** para simular un servidor Apache, colocamos el archivo de desafío en el directorio adecuado:
docker exec -it apache-server bash

```
mkdir -p /usr/local/apache2/htdocs/.well-known/pki-validation/
```

```
Cp /path/to/fileauth.txt /usr/local/apache2/htdocs/.well-known/pki-validation/
```

- La **CA** verificará que el archivo de desafío sea accesible desde la URL:
<http://localhost/.well-known/pki-validation/fileauth.txt>

3. Verificación de la Propiedad del Par de Claves

- El servidor debe firmar el nonce con la clave privada para demostrar que posee la clave privada correspondiente.

```
from cryptography.hazmat.primitives.asymmetric import rsa
from cryptography.hazmat.primitives import serialization
from cryptography.hazmat.primitives.asymmetric import padding
from cryptography.hazmat.primitives import hashes

# Cargar la clave privada
with open("private_server.key", "rb") as private_file:
    private_key = serialization.load_pem_private_key(private_file.read(), password=None)

# El nonce
nonce = b"123456789abcdef"

# Firmar el desafío con la clave privada
signature = private_key.sign(
    nonce,
    padding.PKCS1v15(),
    hashes.SHA256()
)

# Guardar la firma
with open("signed_nonce.sig", "wb") as sig_file:
    sig_file.write(signature)
```

El servidor firmó el nonce con su clave privada, y la CA verificó la firma utilizando la clave pública proporcionada en el CSR.

La IP del servidor es esencial para la verificación del control del dominio, ya que permite a la CA acceder al archivo de desafío.

4. Solicitar el Certificado y Verificación

- Una vez que el archivo de desafío está correctamente colocado y el desafío ha sido firmado, solicita el certificado a la CA.
- La CA verificará que el archivo de desafío sea accesible y que la firma del desafío sea válida. Si ambos son correctos, la CA emitirá el certificado SSL/TLS.
- El certificado se instalará en el servidor web Apache y se configurará para habilitar HTTPS. El servidor debe verificar que el certificado no esté revocado y no esté expirado. Esto se puede hacer utilizando OCSP o CRL.

CONSULTA 3.2. RENOVACIÓN Y REVOCACIÓN DE LOS CERTIFICADOS DIGITALES CUALIFICADOS

3.2.1 RENOVACIÓN DE LOS CERTIFICADOS DIGITALES CUALIFICADOS PERSONALES

El objetivo consiste en gestionar automáticamente la **renovación** de certificados personales antes de que expiren, generando **alertas** y nuevos CSRs.

Proponemos una solución que se encargue de:

1. Detectar expiración de certificados (en formato .pem).
2. Alertar si faltan menos de 30 días.
3. Generar nueva clave RSA de 4096 bits y CSR.
4. Enviar CSR a la AC

María Lobo Iborra
Beatriz Vela Moscoso

5. Recibir nuevo certificado

Para ello, creamos un script en python:

```

1  import os
2  import datetime
3  from OpenSSL import crypto
4
5  def check_cert_expiry(cert_path):
6      with open(cert_path, 'rb') as f:
7          cert = crypto.load_certificate(crypto.FILETYPE_PEM, f.read())
8          exp_date = datetime.datetime.strptime(cert.get_notAfter().decode('ascii'), "%Y%m%d%H%M%S")
9          days_left = (exp_date - datetime.datetime.utcnow()).days
10         return days_left, exp_date
11
12 def generate_renewal_csr(cert_name):
13     key = crypto.PKey()
14     key.generate_key(crypto.TYPE_RSA, 4096)
15
16     req = crypto.X509Req()
17     subj = req.get_subject()
18     subj.CN = cert_name
19     req.set_pubkey(key)
20     req.sign(key, "sha256")
21
22     with open(f"{cert_name}_renewed.key", "wb") as key_file:
23         key_file.write(crypto.dump_privatekey(crypto.FILETYPE_PEM, key))
24
25     with open(f"{cert_name}_renewed.csr", "wb") as csr_file:
26         csr_file.write(crypto.dump_certificate_request(crypto.FILETYPE_PEM, req))
27
28     print(f" Generado CSR para renovación: {cert_name}_renewed.csr")
29
30 # Directorio de certificados
31 certs_dir = "./certificados"
32 for filename in os.listdir(certs_dir):
33     if filename.endswith(".pem"):
34         path = os.path.join(certs_dir, filename)
35         days, exp_date = check_cert_expiry(path)
36         print(f"{filename} expira el {exp_date.date()} ({days} días restantes)")
37         if days < 30:
38             print(f" ALERTA: {filename} caduca en menos de 30 días")
39             cert_base = filename.replace(".pem", "")
40             generate_renewal_csr(cert_base)
41

```

Hemos ejecutado este código con 10 certificados generados aleatoriamente, lo que nos da como respuesta lo siguiente:

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
c:\Users\marie\OneDrive\Escritorio\TERCERO\SCGI\CAI3GRUPAL\renovacion_certificados.py:9: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
  days_left = (exp_date - datetime.datetime.utcnow()).days
cert1.pem expira el 2025-05-24 (9 días restantes)
ALERTA: cert1.pem caduca en menos de 30 días
Generado CSR para renovación: cert1_renewed.csr
cert10.pem expira el 2025-08-22 (99 días restantes)
cert2.pem expira el 2025-06-03 (19 días restantes)
ALERTA: cert2.pem caduca en menos de 30 días
Generado CSR para renovación: cert2_renewed.csr
cert3.pem expira el 2025-06-13 (29 días restantes)
ALERTA: cert3.pem caduca en menos de 30 días
Generado CSR para renovación: cert3_renewed.csr
cert4.pem expira el 2025-06-23 (39 días restantes)
cert5.pem expira el 2025-07-03 (49 días restantes)
cert6.pem expira el 2025-07-13 (59 días restantes)
cert7.pem expira el 2025-07-23 (69 días restantes)
cert8.pem expira el 2025-08-02 (79 días restantes)
cert9.pem expira el 2025-08-12 (89 días restantes)

```


Esta respuesta genera una alerta sobre los certificados que caducan pronto, señalando la fecha en la que caduca y los días restantes. Además, señala la fecha de caducidad y los días restantes del resto de certificados. Además, para aquellos certificados que caducan pronto, genera unos certificados nuevos:

```
> certificados
cert1_renewed.csr
cert1_renewed.key
cert2_renewed.csr
cert2_renewed.key
cert3_renewed.csr
cert3_renewed.key
```

Además, hemos ejecutado el código utilizando certificados digitales reales, comprobando así su correcto funcionamiento:

```
1.bcirt.pem expira el 2026-10-11 (514 días restantes)
2.mcert.pem expira el 2026-06-30 (411 días restantes)
```

3.2.2 REVOCACIÓN DE LOS CERTIFICADOS DIGTALES CUALIFICADOS PERSONALES ANTE LA AUTORIDAD DE CERTIFICACIÓN.

El objetivo es revocar los certificados personales cuando el titular lo solicita, incluso cuando este **ha perdido la clave privada**.

En el caso de tener la clave privada:

Generar una solicitud de revocación con la clave privada es mucho más sencillo que hacerlo sin esta misma, por lo que recomendamos solicitarla **antes** de que se pierda la clave y la solicitud sea necesaria.

Para revocar un certificado digital si ya tenemos nuestra clave privada, bastaría con ejecutar este código en la terminal:

```
openssl ca -revoke cert1.pem -keyfile private_key.pem -cert ca-cert.pem
```

Sin embargo, este código al revocar el certificado digital también lo invalida, de forma que no nos serviría para guardar la solicitud hasta que fuese necesario. Para ello, debemos generar una solicitud y no mandarla a la autoridad de certificación hasta que necesitemos invalidar el certificado digital.

Esto se hace mediante un script en python:

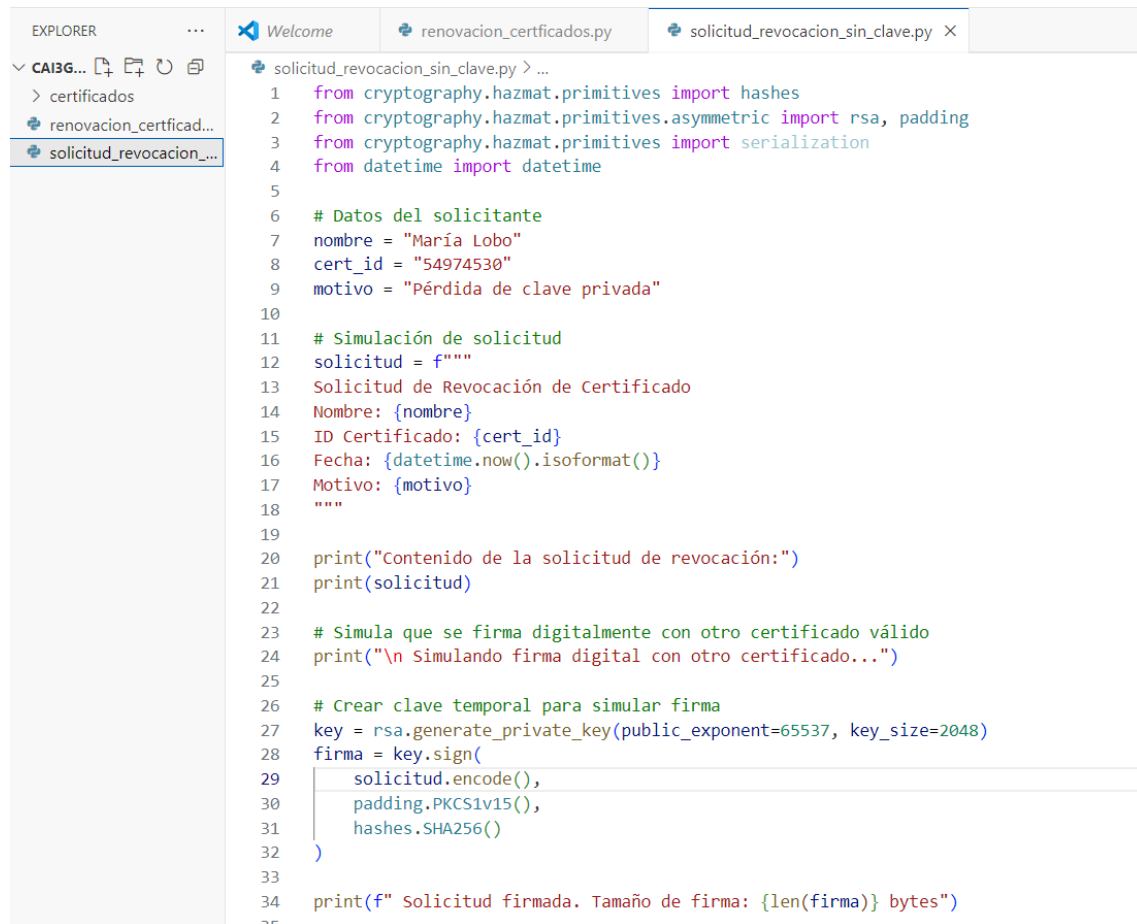
```
solicitud_revocacion.py > ...
1  from cryptography.hazmat.primitives import hashes, serialization
2  from cryptography.hazmat.primitives.asymmetric import rsa, padding
3  from datetime import datetime
4
5  # Datos de la solicitud
6  nombre = "María Pérez"
7  cert_id = "54975531"
8  motivo = "Pérdida de clave privada"
9  fecha = datetime.now().isoformat()
10
11 # Generar el texto de la solicitud
12 solicitud = f"""
13 Solicitud de Revocación de Certificado Digital
14 Nombre: {nombre}
15 ID del Certificado: {cert_id}
16 Fecha de Solicitud: {fecha}
17 Motivo: {motivo}
18 """
19
20 # Guardar la solicitud en archivo de texto
21 with open("solicitud_revocacion.txt", "w", encoding="utf-8") as f:
22     f.write(solicitud)
23
24 print("Solicitud de revocación generada como 'solicitud_revocacion.txt'.")
25
26 # Simular firma digital (opcional)
27 generar_firma = True
28 if generar_firma:
29     # Generar clave simulada (o podrías cargar la tuya real desde archivo)
30     clave_privada = rsa.generate_private_key(public_exponent=65537, key_size=2048)
31
32     firma = clave_privada.sign(
33         solicitud.encode("utf-8"),
34         padding.PKCS1v15(),
35         hashes.SHA256()
36     )
37
38 # Guardar firma binaria y clave privada simulada
39 with open("firma_solicitud.bin", "wb") as f:
40     f.write(firma)
41
42     with open("clave_simulada.pem", "wb") as f:
43         f.write(clave_privada.private_bytes(
44             encoding=serialization.Encoding.PEM,
45             format=serialization.PrivateFormat.TraditionalOpenSSL,
46             encryption_algorithm=serialization.NoEncryption()
47         ))
48
49 print(" Firma digital simulada guardada como 'firma_solicitud.bin'.")
50
```

Con este código se generará una solicitud en formato txt, firmada por el certificado digital actual, y que deberá ser enviada a la CA cuando sea necesario revocar el certificado.

Por otro lado, en caso de no tener la clave privada, tenemos que permitir la revocación mediante:

- Otro certificado válido del titular.
- Autenticación biométrica o presencial.
- Verificación administrativa por la AC o AR.

Para ello, generaremos un script en Python:



```
EXPLORER
CAI3G...
  > certificados
    renovacion_certificad...
    solicitud_revocacion_...

solicitud_revocacion_sin_clave.py > ...
1  from cryptography.hazmat.primitives import hashes
2  from cryptography.hazmat.primitives.asymmetric import rsa, padding
3  from cryptography.hazmat.primitives import serialization
4  from datetime import datetime
5
6  # Datos del solicitante
7  nombre = "María Lobo"
8  cert_id = "54974530"
9  motivo = "Pérdida de clave privada"
10
11 # Simulación de solicitud
12 solicitud = f"""
13 Solicitud de Revocación de Certificado
14 Nombre: {nombre}
15 ID Certificado: {cert_id}
16 Fecha: {datetime.now().isoformat()}
17 Motivo: {motivo}
18 """
19
20 print("Contenido de la solicitud de revocación:")
21 print(solicitud)
22
23 # Simula que se firma digitalmente con otro certificado válido
24 print("\n Simulando firma digital con otro certificado...")
25
26 # Crear clave temporal para simular firma
27 key = rsa.generate_private_key(public_exponent=65537, key_size=2048)
28 firma = key.sign(
29     solicitud.encode(),
30     padding.PKCS1v15(),
31     hashes.SHA256()
32 )
33
34 print(f" Solicitud firmada. Tamaño de firma: {len(firma)} bytes")
```

Y lo que se genera al ejecutar es:



```
PS C:\Users\marie\OneDrive\Escritorio\TERCERO\SCGI\CAI3GRUPAL> & C:/Users/marie/anaconda3/python.exe c:/Users/marie/OneDrive/Escritorio/TERCERO/SCGI/CAI3GRUPAL/solicitud_revocacion_sin_clave.py
Contenido de la solicitud de revocación:

Solicitud de Revocación de Certificado
Nombre: María Lobo
ID Certificado: 54974530
Fecha: 2025-05-14T11:58:50.955971
Motivo: Pérdida de clave privada

Simulando firma digital con otro certificado...
Solicitud firmada. Tamaño de firma: 256 bytes
```

3.2.3 VERIFICACIÓN DEL ESTADO DE REVOCACIÓN DE LOS CERTIFICADOS DE LOS SERVIDORES WEB DEL SERVICIO DE SALUD.

Para verificar si un certificado de **servidor web** ha sido revocado, sin requerir contacto directo del cliente con la AC, proponemos las siguientes soluciones:

1. OCSP (Online Certificate Status Protocol):

El OCSP es un protocolo que permite a un cliente preguntar **en tiempo real** a la AC si un certificado es válido. Para ello, ponemos esto en la terminal:

```
openssl ocsp -issuer ca.pem -cert servidor.pem -url http://ocsp.acme.org
```

2. OCSP Stapling:

El servidor web adjunta la respuesta OCSP a la AC por adelantado, y guarda esa respuesta. Cuando un cliente se conecta, el servidor **adjunta esa respuesta OCSP válida en la conexión TLS**. De esta forma, el cliente no necesita contactar con la AC.

Para ello, el servidor consulta a la AC cada cierto tiempo, guarda la respuesta OCSP firmada, y cuando el cliente se conecta por HTTPS el servidor le adjunta esa respuesta OCSP automáticamente.

Para ello, realizamos esta configuración en NGINX:

```
server {  
  
    listen 443 ssl;  
  
    server_name www.saludpublica.org;  
  
    ssl_certificate /etc/ssl/certs/server.crt;  
    ssl_certificate_key /etc/ssl/private/server.key;  
    ssl_trusted_certificate /etc/ssl/certs/ca-chain.pem;  
  
    ssl_stapling on;  
    ssl_stapling_verify on;  
  
    resolver 8.8.8.8 1.1.1.1 valid=300s;  
    resolver_timeout 5s;  
}
```

Es una configuración **segura, moderna y eficiente**, ideal para un entorno sanitario donde la **privacidad y la velocidad de validación** son importantes.

3. CRL (Certificate Revocation List):

La CRL es un archivo que publica periódicamente la AC con los **números de serie de los certificados revocados**. Lo que hacemos es descargarla desde la AC y verificarla:

```
wget http://ac.miac.com/crl/lista.crl  
  
openssl crl -in crl_lista.crl -text
```

Método	¿Tiempo real?	¿Privado?	¿Rápido?	¿Fácil de implementar?	¿Recomendado?
OCSP	Sí	No	No	Fácil para clientes	Depende del entorno
OCSP Stapling	Sí	Sí	Sí	Requiere configurar el servidor	Sí, ideal
CRL	No	Sí	No	Sencillo pero manual	No, solo como respaldo

En un sistema público de salud, donde la **eficiencia, privacidad y seguridad** son fundamentales, la **mejor opción es OCSP Stapling**. Permite a los servidores web garantizar que sus certificados siguen siendo válidos **sin que los pacientes o profesionales tengan que contactar con la Autoridad de Certificación**.

CONSULTA 3.3 UTILIZACIÓN DE LOS CERTIFICADOS DIGITALES PERSONALES PARA AUTENTICARSE ANTE EL SERVICIO DE SALUD Y PARA FIRMAR DOCUMENTOS XML Y PDF.

3.3.1 UTILIZACIÓN DE LOS CERTIFICADOS DIGITALES CUALIFICADOS PERSONALES PARA AUTENTICARSE ANTE EL SERVIDIO DE SALUD.

1. Generación del Nonce por el Servidor

- El servidor genera un nonce que será firmado por el cliente para autenticarlo. Un nonce es un número o cadena aleatoria que se utiliza para asegurarse de que la firma corresponde a una operación reciente (y no a una reutilización de una firma anterior). El servidor lo generará y lo enviará al cliente para que lo firme.

```
import os
```

```
# Generar un nonce aleatorio
nonce = os.urandom(16).hex()
print("Nonce generado:", nonce)
```

|

- Aquí generamos un nonce aleatorio de 16 bytes que el servidor enviará al cliente.

2. Firma del Nonce por el Cliente

- El cliente firma el nonce con su clave privada para demostrar que posee la clave privada asociada al certificado digital cualificado. A continuación, se muestra cómo hacerlo en Python usando la librería cryptography.

```
from cryptography.hazmat.primitives.asymmetric import rsa
from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives.asymmetric import padding
from cryptography.hazmat.primitives import serialization

# Cargar la clave privada (en formato PEM) del cliente
with open("private_key.pem", "rb") as key_file:
    private_key = serialization.load_pem_private_key(key_file.read(), password=None)

# El nonce recibido del servidor
nonce = "nonce_generado_por_el_servidor"

# Firmar el nonce con la clave privada
signature = private_key.sign(
    nonce.encode(),
    padding.PKCS1v15(),
    hashes.SHA256()
)

# Guardar la firma en un archivo
with open("signed_nonce.sig", "wb") as sig_file:
    sig_file.write(signature)

print("Nonce firmado y guardado.")
```

- El cliente carga su clave privada desde el archivo `private_key.pem`. Luego firma el nonce con la clave privada utilizando PKCS1v15 y SHA-256. La firma resultante se guarda en el archivo `signed_nonce.sig`, que será enviado de vuelta al servidor.

3. Verificación de la Firma en el Servidor

- El **servidor** necesita verificar que el **cliente** posee la **clave privada** asociada al **certificado digital**. Para ello, el servidor utiliza la **clave pública** del cliente (que puede estar en el **certificado digital** del cliente) para verificar que la firma es válida. Aquí está el código para realizar la **verificación** de la firma:

```
from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives.asymmetric import padding
from cryptography.hazmat.primitives import serialization
from cryptography.hazmat.primitives.asymmetric import rsa

# Cargar el certificado público del cliente
with open("public_key.pem", "rb") as cert_file:
    public_key = serialization.load_pem_public_key(cert_file.read())

# Cargar el nonce y la firma del cliente
with open("signed_nonce.sig", "rb") as sig_file:
    signature = sig_file.read()

nonce = "nonce_generado_por_el_servidor"

# Verificar la firma con la clave pública
try:
    public_key.verify(
        signature,
        nonce.encode(),
        padding.PKCS1v15(),
        hashes.SHA256()
    )
    print("Firma verificada exitosamente.")
except Exception as e:
    print(f"Error al verificar la firma: {e}")
```

- El servidor carga la clave pública del cliente desde el archivo public_key.pem. Luego verifica la firma utilizando el nonce original y la firma que se recibió. Si la firma es válida, significa que el cliente posee la clave privada asociada al certificado digital.

4. Verificación de la Validez del Certificado (Expiración y Revocación).

- El servidor debe asegurarse de que el certificado del cliente no haya expirado. Esto se puede verificar cargando el certificado y revisando su fecha de expiración.

```
from cryptography import x509
from cryptography.hazmat.backends import default_backend

# Cargar el certificado del cliente
with open("client_cert.pem", "rb") as cert_file:
    cert = x509.load_pem_x509_certificate(cert_file.read(), default_backend())

# Verificar la fecha de expiración
if cert.not_valid_after < datetime.datetime.now():
    print("El certificado está expirado.")
else:
    print("El certificado es válido.")
```

Prueba

final

consulta:

```
PS C:\Users\Beatriz Vela Moscoso\Downloads\CAI3.1\consulta3.3> cd .\consulta3.3.1\
PS C:\Users\Beatriz Vela Moscoso\Downloads\CAI3.1\consulta3.3\consulta3.3.1> python nonce.py
Nonce generado: 3cc464abe692dc29a50203edd4862758
PS C:\Users\Beatriz Vela Moscoso\Downloads\CAI3.1\consulta3.3\consulta3.3.1> python claves.py
Clave privada y clave pública generadas correctamente.
PS C:\Users\Beatriz Vela Moscoso\Downloads\CAI3.1\consulta3.3\consulta3.3.1> python nonce2.py
Nonce firmado y guardado.
PS C:\Users\Beatriz Vela Moscoso\Downloads\CAI3.1\consulta3.3\consulta3.3.1> python verificarfirma.py
Firma verificada exitosamente.
PS C:\Users\Beatriz Vela Moscoso\Downloads\CAI3.1\consulta3.3\consulta3.3.1> █
```

Esta prueba de concepto demuestra cómo se puede implementar una autenticación passwordless en el Servicio de Salud, utilizando certificados digitales cualificados para autenticar a los usuarios sin necesidad de contraseñas. El proceso incluye la firma de un nonce por parte del cliente, la verificación de la firma por el servidor, y la verificación de la validez del certificado.

3.3.2. Utilización de certificados digitales para firmas documentos PDF y XML y verificación de la firma digital

Con el objetivo de digitalizar los procesos de validación documental dentro del Servicio de Salud, recomendamos el uso de la herramienta **AutoFirma**, desarrollada y mantenida por el Gobierno de España. Se trata de un programa gratuito, oficial y compatible con certificados digitales cualificados, ideal para firmar y verificar archivos en formato **PDF** o **XML**, entre otros.

AutoFirma es una aplicación que permite a los empleados del Servicio de Salud firmar digitalmente documentos de forma legal, segura y sencilla utilizando sus certificados personales cualificados (FNMT, DNle, ACCV, etc.), disponible para Windows, macOS y Linux.

Proceso:

El proceso que deben seguir los empleados del Servicio de Salud es el siguiente:

1. Descargar la aplicación de **AutoFirma** a través de la página oficial del Ministerio de Asuntos Económicos y Transformación Digital.

2. **Preparar el archivo a firmar** (PDF o XML).
3. **Abrir AutoFirma** e ir a la opción **“Firmar archivo”**.
4. **Seleccionar el archivo** y confirmar el uso del certificado digital personal cualificado.
5. **Elegir el formato de firma:**
 - En PDF: firma visible o invisible integrada.
 - En XML: firma envolvente, detached o enveloped, según el esquema requerido.
6. **Finalizar el proceso**, lo que generará un nuevo archivo firmado digitalmente.

Este procedimiento permite garantizar la **autenticidad** (quién ha firmado) y la **integridad** (que el archivo no ha sido modificado desde la firma) del documento.

Además, AutoFirma también permite **verificar la validez de una firma digital**. Para ello, los usuarios pueden:

1. Abrir AutoFirma y acceder a la opción **“Verificar firma”**.
2. Seleccionar el archivo firmado (por ejemplo, un PDF o XML).
3. Visualizar los resultados de la validación, que incluyen:
 - Identidad del firmante (nombre, DNI/NIE).
 - Fecha y hora de la firma.
 - Resultado de la comprobación de integridad del documento.
 - Estado de revocación del certificado.

Este proceso permite confirmar que la firma fue realizada por la persona correcta, en el momento indicado, y que el documento no ha sido alterado.

Característica	Firma PDF	Firma XML
Formato de archivo	Documento visual, destinado a ser leído por personas	Archivo estructurado para ser leído por máquinas
Firma visible	Puede ser visible (una marca en el documento)	No es visible al abrir el XML, es un bloque de datos firmado
Dónde se inserta la firma	Se incrusta en el archivo como una capa de seguridad	Se añade una etiqueta XML de firma (<ds:Signature>)

Estándar de firma	PDF Advanced Electronic Signatures (PAdES)	XML Digital Signature (XAdES)
Herramientas para ver la firma	Adobe Acrobat, AutoFirma, navegadores PDF	AutoFirma, herramientas XML, validadores XAdES
Casos de uso típicos	Informes, contratos, certificados, consentimientos	Facturas electrónicas, historiales médicos estructurados, mensajes firmados
Validación	Suele mostrar aviso visual al abrir el PDF	Requiere herramienta que entienda firmas XML

Conclusiones

La adopción de esta solución no solo refuerza la seguridad del Servicio de Salud, sino que también mejora la eficiencia operativa al reducir la dependencia de contraseñas, eliminando así los riesgos asociados al uso de contraseñas tradicionales.

Finalmente, este enfoque de autenticación passwordless sienta las bases para futuras mejoras en la gestión de la identidad digital en entornos críticos como el del Servicio de Salud, contribuyendo a crear un entorno digital más seguro, accesible y confiable.

Realizado por:

María Lobo Iborra
Beatriz Vela Moscoso